

#20001. Deepseek Sparse Attention Indexer Score BF16

未公开

传统

20000 ms

4096 MiB

1740
通过

2492
提交

提交

题目描述

你需要实现 DeepSeek Sparse Attention (DSA) 中的 **BF16 Indexer Scores** 算子:

```
scores = dsa_indexer_scores_bf16(q_idx, k_idx_cache, w_idx, block_table, context_lens)
```

该算子用于 decode 模式: 每个 batch item 只有一个当前 query token, 需要计算该 query 对所有可见 context token 的 indexer score. 输入的 query indexer 向量、paged key indexer cache 和 indexer head 权重均为 BF16; 内部 dot product 和 score accumulation 建议使用 FP32; 最终输出写回 BF16。

评测程序会将你的 CUDA 源码编译为动态链接库, 并在 GPU 上通过 FFI 调用你导出的 run_kernel 函数。评测会同时进行正确性校验和性能计时。

本题目标评测硬件为 **NVIDIA H800 GPU**. 在提交时, 你也应该选择该硬件。你可以利用 Hopper 架构上的硬件特性进行优化, 但这不是正确性要求; 基础实现可以使用常规 CUDA kernel、warp-level reduction、shared memory 和向量化 load 等方法完成。

计算定义

设:

```
MaxSeqLen = MaxPages * PageSize
```

输入张量含义如下:

- q_idx: BF16, shape (B, Hidx, Didx)
- k_idx_cache: BF16, shape (NumPages, PageSize, Didx)
- w_idx: BF16, shape (B, Hidx)
- block_table: INT32, shape (B, MaxPages)
- context_lens: INT32, shape (B)
- scores: BF16, shape (B, MaxSeqLen), 输出缓冲区

对每个 batch item b 和 logical context token s, 如果:

```
0 ≤ s < context_lens[b]
```

则需要计算:

```
scores[b, s] =
    sum_{j=0}^{Hidx-1}
        float(w_idx[b, j]) *
        ReLU(dot(q_idx[b, j, :], k_idx[b, s, :]))
```

其中:

```
dot(q, k) = sum_{d=0}^{Didx-1} float(q[d]) * float(k[d])
ReLU(x) = max(x, 0)
```

如果:

```
s ≥ context_lens[b]
```

则必须写入 BF16 表示的 negative infinity:

```
scores[b, s] = -inf
```

注意, 输出缓冲区在调用前可能含有任意旧值。无效位置不能跳过, 必须显式写成 -inf。

Paged Cache 地址计算

k_idx_cache 使用 paged cache 布局, logical token index s 不能直接当作 physical cache offset 使用, 必须通过 block_table 转换。

对 batch b 中的 logical token s:

```
logical_page = s / PageSize
page_offset  = s % PageSize
physical_page = block_table[b, logical_page]
```

于是:

```
k_idx[b, s, d] = k_idx_cache[physical_page, page_offset, d]
```

评测数据可能使用非连续、乱序的 block_table。不要假设:

```
physical_page == logical_page
```

评测保证所有会被访问的 block_table[b, logical_page] 都是合法 physical page id。

数据布局

所有张量均为连续 row-major 布局, 不需要支持任意 stride。

线性 offset 如下:

```
q_idx[b, j, d]
    → ((b * Hidx + j) * Didx + d)

k_idx_cache[p, o, d]
    → ((p * PageSize + o) * Didx + d)
```

```
w_idx[b, j]
    → (b * Hidx + j)

block_table[b, logical_page]
    → (b * MaxPages + logical_page)

context_lens[b]
    → b

scores[b, s]
    → (b * MaxSeqLen + s)
```

接口约定

CUDA	Triton	TileLang
------	--------	----------

你必须在提交的 CUDA 源码中提供如下 C 符号。函数名、参数类型、参数顺序必须完全一致，并使用 extern "C" 防止 C++ name mangling。

```
#include <stdint.h>
#include <cuda_bf16.h>

extern "C" void run_kernel(
    const __nv_bfloat16* q_idx,
    const __nv_bfloat16* k_idx_cache,
    const __nv_bfloat16* w_idx,
    const int32_t* block_table,
    const int32_t* context_lens,
    __nv_bfloat16* scores,
    int64_t B,
    int64_t Hidx,
    int64_t Didx,
    int64_t MaxPages,
    int64_t PageSize
);
```

参数说明：

- q_idx: 输入指针，BF16, shape (B, Hidx, Didx), 只读
- k_idx_cache: 输入指针，BF16, shape (NumPages, PageSize, Didx), 只读
- w_idx: 输入指针，BF16, shape (B, Hidx), 只读
- block_table: 输入指针，INT32, shape (B, MaxPages), 只读
- context_lens: 输入指针，INT32, shape (B), 只读
- scores: 输出指针，BF16, shape (B, MaxSeqLen), 必须写入结果
- B: batch size
- Hidx: indexer head 数
- Didx: indexer 向量维度
- MaxPages: 每个 batch item 的最大 logical page 数
- PageSize: 每个 page 中的 token 数

NumPages 是评测数据中实际分配的 physical page 总数，不作为参数传入。你的实现不需要枚举 NumPages，只需要使用 block_table 给出的合法 physical page id 访问 k_idx_cache。

run_kernel 内部需要完成以下工作：

- 根据输入尺寸计算合适的 CUDA launch 配置
- launch 你实现的 CUDA kernel
- 不要在 run_kernel 中打印调试信息
- 不要读写题面未要求的外部文件

输入格式

本题的输入由评测程序在 GPU 上构造，并按接口顺序传入 run_kernel。标准输入只用于评测系统选择测试点，用户代码不需要读取标准输入。

评测程序传入：

- q_idx: BF16 CUDA tensor, shape (B, Hidx, Didx)
- k_idx_cache: BF16 CUDA tensor, shape (NumPages, PageSize, Didx)
- w_idx: BF16 CUDA tensor, shape (B, Hidx)
- block_table: INT32 CUDA tensor, shape (B, MaxPages)
- context_lens: INT32 CUDA tensor, shape (B)
- scores: BF16 CUDA tensor, shape (B, MaxSeqLen), 输出缓冲区
- B, Hidx, Didx, MaxPages, PageSize: int64 标量

输出格式

你不需要向标准输出打印任何内容。你需要把结果写入 scores。

对所有有效位置：

```
scores[b, s] = bf16(
    sum_j float(w_idx[b, j]) *
        max(sum_d float(q_idx[b, j, d]) *
            float(k_idx_cache[physical_page(s), page_offset(s), d]),
            0)
)
```

对所有无效位置：

```
scores[b, s] = bf16(-inf)
```

样例

设：

```
B = 1
Hidx = 64
Didx = 128
```

```
MaxPages = 1
PageSize = 4
MaxSeqLen = 4
```

若:

```
context_lens[0] = 2
block_table[0, 0] = 0
```

则输出 scores 的 shape 为 (1, 4)。对当前 decode query，只有 s = 0, 1 是有效位置，s = 2, 3 必须写成 -inf：

```
scores[0, 0] = valid score
scores[0, 1] = valid score
scores[0, 2] = -inf
scores[0, 3] = -inf
```

测试用例尺寸与正确性要求

评测使用固定的 10 组 testcase。所有 testcase 中 Hidx = 64、Didx = 128、PageSize = 64，其中 Hidx 和 Didx 对应 DeepSeek-V3.2 indexer 的真实参数。并且：

```
MaxSeqLen = MaxPages * PageSize
```

测试用例 ID	B	Hidx	Didx	MaxPages	PageSize	MaxSeqLen	scores shape
1	1	64	128	8	64	512	(1, 512)
2	2			16		1024	(2, 1024)
3	4			32		2048	(4, 2048)
4	8			64		4096	(8, 4096)
5	16			128		8192	(16, 8192)
6	32						(32, 8192)
7				256		16384	(32, 16384)
8	64			128		8192	(64, 8192)
9				256		16384	(64, 16384)
10				512		32768	(64, 32768)

NumPages 不作为 run_kernel 参数传入，由评测数据生成器根据 block_table 和 cache 分配策略确定。你的实现只应通过 block_table[b, logical_page] 取得 physical page id。

正确性要求：

- 所有有效位置必须与 reference 在 BF16 允许误差范围内一致 (相对误差或绝对误差不超过0.01)
- 所有无效位置必须写入 -inf
- 必须通过 block_table 读取 k_idx_cache
- 必须支持 context_lens[b] == 0
- 不能读取越界的 block_table、q_idx、k_idx_cache、w_idx
- 不能写越界的 scores
- 不能假设所有 batch item 的 context_lens 相同
- 不能假设 block_table[b, p] == p
- 不能硬编码 B、Hidx、Didx、MaxSeqLen

评分细则

本实验总分由正确性分数、性能分数和实验报告三部分组成。正确性分数占本次作业总分的40%，性能分数占本次作业的50%，实验报告分数占本次作业总分的10%

正确性部分包含数值检查。为避免直接提交朴素 baseline，本题要求提交实现相对 OJ baseline 的加速比 **大于 2**。如果输出正确但加速比不超过 2，则不能获得正确性分，也不会进入性能评分。测试点 1 主要用于检查基本接口和边界行为，**不要求达到 2 倍加速比**，只要数值正确即可获得该测试点的正确性分。

性能分数的评分标准为：每个测试点为**等权重**分布，即每个测试点的性能分数占本次作业总分的5%。对于每个测试点，其性能分数按照如下方法计算：性能排名前 10% 的同学得到 100% 的分数，排名 10% — 20% 的同学得到 90% 的分数，依此类推。对于任何测试用例，获得正确性分数的同学将至少获得 10% 的性能分数。

以最后一次提交为准

OJ Baseline性能如下：

测试用例 ID	OJ baseline 参考耗时
1	8 us
2	1165 us
3	1170 us
4	1179 us
5	4551 us
6	9 ms
7	18 ms
8	
9	35 ms
10	67 ms

对于实验报告，你需要以PDF格式在网络学堂上提交。内容包括：

- 介绍你的实现方法，包括有哪些 GPU kernel、每个 kernel 中线程和线程块如何分配、如何利用各级存储等（不要直接粘贴代码），如果你的Kernel使用了新的GPU指令/优化技术（如WGMMMA等），请在实验报告中介绍；
- 给出你的实现在测试点5、7、10下的运行时间以及相对于CUDA Baseline的加速比。
- （如有）描述你使用的AI模型、AI工具，以及你是如何在本次作业中使用AI的。

CUDA Baseline 代码

下面给出一个可编译的 CUDA baseline。它优先保证正确性，性能不是目标。你可以以此为起点逐步优化。

```
#include <stdint.h>
#include <math.h>
#include <cuda_runtime.h>
#include <cuda_bf16.h>

__global__ void dsa_indexer_scores_baseline_kernel(
    const __nv_bfloat16* __restrict__ q_idx,
    const __nv_bfloat16* __restrict__ k_idx_cache,
    const __nv_bfloat16* __restrict__ w_idx,
    const int32_t* __restrict__ block_table,
    const int32_t* __restrict__ context_lens,
    __nv_bfloat16* __restrict__ scores,
    int64_t B,
    int64_t Hidx,
    int64_t Didx,
    int64_t MaxPages,
    int64_t PageSize
) {
    const int64_t MaxSeqLen = MaxPages * PageSize;
    const int64_t total = B * MaxSeqLen;

    for (int64_t linear = blockIdx.x * blockDim.x + threadIdx.x;
         linear < total;
         linear += (int64_t)blockDim.x * gridDim.x) {
        const int64_t s = linear % MaxSeqLen;
        const int64_t b = linear / MaxSeqLen;

        const int64_t score_offset = b * MaxSeqLen + s;
        const int32_t visible = context_lens[b];

        if (s >= (int64_t)visible) {
            scores[score_offset] = __float2bfloat16(-INFINITY);
            continue;
        }

        const int64_t logical_page = s / PageSize;
        const int64_t page_offset = s - logical_page * PageSize;
        const int32_t physical_page = block_table[b * MaxPages + logical_page];

        float acc = 0.0f;

        for (int64_t j = 0; j < Hidx; ++j) {
            const int64_t q_base = (b * Hidx + j) * Didx;
            const int64_t k_base = ((int64_t)physical_page * PageSize + page_offset) * Didx;

            float dot = 0.0f;
            for (int64_t d = 0; d < Didx; ++d) {
                const float q = __bfloat162float(q_idx[q_base + d]);
                const float k = __bfloat162float(k_idx_cache[k_base + d]);
                dot += q * k;
            }

            if (dot > 0.0f) {
                const float w = __bfloat162float(w_idx[b * Hidx + j]);
                acc += w * dot;
            }
        }

        scores[score_offset] = __float2bfloat16(acc);
    }
}

extern "C" void run_kernel(
    const __nv_bfloat16* q_idx,
    const __nv_bfloat16* k_idx_cache,
    const __nv_bfloat16* w_idx,
    const int32_t* block_table,
    const int32_t* context_lens,
    __nv_bfloat16* scores,
    int64_t B,
    int64_t Hidx,
    int64_t Didx,
    int64_t MaxPages,
    int64_t PageSize
) {
    const int64_t MaxSeqLen = MaxPages * PageSize;
    const int64_t total = B * MaxSeqLen;
    if (total <= 0) {
        return;
    }

    const int threads = 256;
    int64_t blocks64 = (total + threads - 1) / threads;
    if (blocks64 > 65535) {
        blocks64 = 65535;
    }
    const int blocks = (int)blocks64;

    dsa_indexer_scores_baseline_kernel<<<blocks, threads>>>(
        q_idx,
        k_idx_cache,
        w_idx,
        block_table,
        context_lens,
        scores,
        B,
        Hidx,
        Didx,
        MaxPages,
        PageSize
    );
}
```

```
    maxPages,  
    PageSize  
};  
}
```